

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 10 Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Shaik Rafiqhudddin (192525129) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

QUESTION 1

Problem Statement

An airport has five check-in counters serving passengers. Each passenger arrival is modelled as a process with a defined arrival time and service (burst) time. During peak hours, passengers requiring special assistance are given priority. Analyse the scheduling using FCFS and Priority Scheduling; determine the waiting time and turnaround time for each passenger, and identify which algorithm gives better service efficiency.

Aim

To model airport check-in as a CPU scheduling problem and compare FCFS and Priority Scheduling in terms of waiting time, turnaround time, and fairness to special-assistance passengers.

Assumptions

Since exact arrival/service times were not specified, the following realistic data set of six passengers is assumed for one check-in counter during a peak-hour window.

Priority 1 = highest (special-assistance passenger); Priority 4 = lowest.

Given Data

Passenger (Process)	Arrival Time (min)	Service/Burst Time (min)	Priority
P1	0	4	3
P2 (Special Assistance)	1	3	1
P3	2	2	4
P4	3	6	2
P5 (Special Assistance)	4	5	1
P6	5	3	3

First-Come, First-Served (FCFS) Scheduling

FCFS is a non-preemptive CPU scheduling algorithm in which the process that requests the CPU first is allocated the CPU first. Processes are executed strictly in the order of their arrival, using a simple FIFO (First-In-First-Out) ready queue.

Working Principle: When the CPU becomes free, the process at the head of the ready queue is dispatched and runs to completion (or until it needs I/O) before the next process in the queue is selected. No process can pre-empt a running process.

Advantages: Simple to understand and implement; Fair in the sense that every process is served in arrival order; No starvation, since every process is eventually served.

Disadvantages: Leads to the convoy effect, where short processes wait behind long ones; Average waiting time is generally high compared to SJF; Not suitable for time-sharing / interactive systems.

Applications: Batch processing systems; Print queues and simple task queues; Systems where arrival order fairness matters more than turnaround time.

Step-by-Step Solution — FCFS

Passengers are served strictly in order of arrival: P1, P2, P3, P4, P5, P6.

Gantt Chart — FCFS

P1	P2	P3	P4	P5	P6
0-4	4-7	7-9	9-15	15-20	20-23

Calculations — FCFS (Completion Time = previous CT or Arrival, whichever is later, + Burst)

Passenger	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P1	0	4	4	4	0
P2	1	3	7	6	3
P3	2	2	9	7	5
P4	3	6	15	12	6
P5	4	5	20	16	11
P6	5	3	23	18	15

Average Waiting Time = $(0+3+5+6+11+15)/6 = 40/6 = 6.67$ min

Average Turnaround Time = $(4+6+7+12+16+18)/6 = 63/6 = 10.5$ min

Priority Scheduling

Priority Scheduling assigns each process a priority number; the CPU is allocated to the process with the highest priority (in this document, a lower numeric value denotes a higher priority). This implementation is non-preemptive unless stated otherwise.

Working Principle: The scheduler always picks the highest-priority process among those that have arrived and are waiting. Processes with equal priority are served in FCFS order. A preemptive variant interrupts a running lower-priority process the instant a higher-priority process arrives.

Advantages: Critical or time-sensitive tasks receive prompt service; Flexible — priority can reflect urgency, resource needs, or business rules.

Disadvantages: Low-priority processes may suffer indefinite starvation; Requires a fair method (e.g., aging) to bound waiting time.

Applications: Emergency/critical-task handling (hospitals, satellites); Real-time and embedded systems; Multilevel service-tier systems (e.g., premium vs standard users).

Step-by-Step Solution — Priority Scheduling (non-preemptive, ties broken by earlier arrival)

Dispatch order determined by lowest priority number among arrived passengers at each decision point: P1 → P2 → P5 → P4 → P6 → P3.

Gantt Chart — Priority Scheduling

P1	P2	P5	P4	P6	P3
0-4	4-7	7-12	12-18	18-21	21-23

Calculations — Priority Scheduling

Passenger	Priority	AT	BT	CT	TAT	WT
P1	3	0	4	4	4	0
P2	1	1	3	7	6	3
P5	1	4	5	12	8	3
P4	2	3	6	18	15	9
P6	3	5	3	21	16	13
P3	4	2	2	23	21	19

Average Waiting Time = $(0+3+3+9+13+19)/6 = 47/6 = 7.83$ min

Average Turnaround Time = $(4+6+8+15+16+21)/6 = 70/6 = 11.67$ min

Comparison Table

Parameter	FCFS	Priority Scheduling
Average Waiting Time	6.67 min	7.83 min
Average Turnaround Time	10.5 min	11.67 min
Waiting time of special-assistance passengers (P2, P5)	3, 11	3, 3
Fairness	High (strict order)	Favours priority passengers
Starvation risk	None	Low-priority passengers (e.g. P3) may wait long

Result

FCFS gives a marginally better overall average waiting/turnaround time, but it treats every passenger identically regardless of need. Priority Scheduling raises the overall average slightly, but dramatically reduces the waiting time of special-assistance passengers (P5's wait falls from 11 to 3 minutes).

Conclusion

For an airport during peak hours, Priority Scheduling provides better service efficiency where it matters — special-assistance passengers are processed promptly — even though the crude system-wide average is slightly higher. A hybrid approach (priority scheduling with periodic aging for regular passengers) is recommended to combine responsiveness for priority passengers with fairness for everyone else.

QUESTION 2

Problem Statement

A hospital emergency room receives patients continuously. Critical patients must be treated immediately, while non-critical patients wait in a queue. Model this as a process scheduling problem using Priority Scheduling and Round Robin (time quantum = 5 minutes). Compare the response time for critical and non-critical patients.

Aim

To evaluate how Priority Scheduling and Round Robin affect response time for critical versus non-critical patients in an emergency room.

Assumptions

Six patients arrive over a short peak window; Priority 1 = Critical, Priority 2/3 = Non-critical (2 = urgent, 3 = routine).

Response Time = time from arrival until the patient is first attended to (first CPU allocation).

Given Data

Patient	Type	Arrival (min)	Treatment/Burst (min)	Priority
P1	Critical	0	8	1
P2	Non-critical	1	6	3
P3	Critical	2	4	1
P4	Non-critical	3	10	3
P5	Non-critical	4	5	2
P6	Critical	6	3	1

Priority Scheduling

Priority Scheduling assigns each process a priority number; the CPU is allocated to the process with the highest priority (in this document, a lower numeric value denotes a higher priority). This implementation is non-preemptive unless stated otherwise.

Working Principle: The scheduler always picks the highest-priority process among those that have arrived and are waiting. Processes with equal priority are served in FCFS order. A

preemptive variant interrupts a running lower-priority process the instant a higher-priority process arrives.

Advantages: Critical or time-sensitive tasks receive prompt service; Flexible — priority can reflect urgency, resource needs, or business rules.

Disadvantages: Low-priority processes may suffer indefinite starvation; Requires a fair method (e.g., aging) to bound waiting time.

Applications: Emergency/critical-task handling (hospitals, satellites); Real-time and embedded systems; Multilevel service-tier systems (e.g., premium vs standard users).

Step-by-Step Solution — Priority Scheduling (non-preemptive)

Dispatch order: P1 → P3 → P6 → P5 → P2 → P4 (ties among priority-1 patients broken by earlier arrival).

P1	P3	P6	P5	P2	P4
0-8	8-12	12-15	15-20	20-26	26-36

Response Time — Priority Scheduling

Patient	Type	Arrival	Start (Response)	Response Time
P1	Critical	0	0	0
P3	Critical	2	8	6
P6	Critical	6	12	6
P5	Non-critical	4	15	11
P2	Non-critical	1	20	19
P4	Non-critical	3	26	23

Average Response Time — Critical = $(0+6+6)/3 = 4$ min. Average Response Time — Non-critical = $(11+19+23)/3 = 17.67$ min.

Round Robin (RR) Scheduling (Time Quantum = 5 min)

Round Robin is a preemptive scheduling algorithm designed for time-sharing systems. Each process is given a fixed time slice (quantum); if it does not finish within that slice, it is preempted and placed at the back of the ready queue.

Working Principle: The ready queue is treated as a circular FIFO queue. The CPU executes the process at the head of the queue for at most one quantum. If the process needs more time, it is moved to the tail of the queue and the next process is dispatched.

Advantages: Fair allocation of CPU time to every process; Good response time, well suited to interactive/time-shared systems; No starvation.

Disadvantages: Performance is highly sensitive to the size of the time quantum; Higher average turnaround time than SJF for CPU-bound jobs; Increased context-switching overhead for small quanta.

Applications: Time-sharing operating systems; Interactive terminals and multi-user cloud task queues; Traffic-signal and round-trip resource-sharing systems.

Step-by-Step Solution — Round Robin (Quantum = 5 min, all patients treated equally regardless of criticality)

P1	P2	P3	P4	P5	P1	P6	P2	P4
0-5	5-10	10-14	14-19	19-24	24-27	27-30	30-31	31-36

Response Time — Round Robin

Patient	Type	Arrival	First Start	Response Time
P1	Critical	0	0	0
P2	Non-critical	1	5	4
P3	Critical	2	10	8
P4	Non-critical	3	14	11
P5	Non-critical	4	19	15
P6	Critical	6	27	21

Average Response Time — Critical = $(0+8+21)/3 = 9.67$ min. **Average Response Time — Non-critical** = $(4+11+15)/3 = 10$ min.

Comparison Table

Parameter	Priority Scheduling	Round Robin (q=5)
Avg. response — Critical patients	4 min	9.67 min
Avg. response — Non-critical patients	17.67 min	10 min

Parameter	Priority Scheduling	Round Robin (q=5)
Fairness	Low for non-critical	High — equal share
Starvation risk	Non-critical patients may starve	None
Suitability for ER	Best for critical care	Best for routine/general queue

Result

Priority Scheduling gives critical patients a much faster average response (4 min vs 9.67 min under RR), at the cost of longer waits for non-critical patients. Round Robin is fairer overall but delays urgent care.

Conclusion

An emergency room should use (preemptive) Priority Scheduling as the primary policy so that critical patients are always treated first, while Round Robin (or a lower-priority RR sub-queue) can be applied among non-critical patients to keep their wait times fair and bounded.

QUESTION 3

Problem Statement

A cloud service provider receives ten computational tasks from clients. Each task requires different CPU burst times and arrives at different intervals. Apply FCFS, SJF, and Round Robin scheduling to determine the average waiting time and CPU utilization. Recommend the best scheduling algorithm for the cloud environment.

Aim

To compare FCFS, SJF and Round Robin for ten cloud computation tasks and recommend the most suitable algorithm.

Assumptions

Ten tasks with staggered arrivals and varying CPU burst times, given below. Round Robin quantum assumed = 4 ms (a typical cloud time-slice).

All tasks are CPU-bound (no I/O modelled), so total CPU-busy time equals the sum of burst times in every case.

Given Data

Task	Arrival (ms)	Burst (ms)
T1	0	5
T2	1	3
T3	2	8
T4	3	6
T5	4	2
T6	5	4
T7	6	9
T8	7	1
T9	8	7
T10	9	3

Total burst time = $5+3+8+6+2+4+9+1+7+3 = 48$ ms.

First-Come, First-Served (FCFS) Scheduling

FCFS is a non-preemptive CPU scheduling algorithm in which the process that requests the CPU first is allocated the CPU first. Processes are executed strictly in the order of their arrival, using a simple FIFO (First-In-First-Out) ready queue.

Working Principle: When the CPU becomes free, the process at the head of the ready queue is dispatched and runs to completion (or until it needs I/O) before the next process in the queue is selected. No process can pre-empt a running process.

Advantages: Simple to understand and implement; Fair in the sense that every process is served in arrival order; No starvation, since every process is eventually served.

Disadvantages: Leads to the convoy effect, where short processes wait behind long ones; Average waiting time is generally high compared to SJF; Not suitable for time-sharing / interactive systems.

Applications: Batch processing systems; Print queues and simple task queues; Systems where arrival order fairness matters more than turnaround time.

Calculations — FCFS

Task	AT	BT	CT	TAT	WT
T1	0	5	5	5	0
T2	1	3	8	7	4
T3	2	8	16	14	6
T4	3	6	22	19	13
T5	4	2	24	20	18
T6	5	4	28	23	19
T7	6	9	37	31	22
T8	7	1	38	31	30
T9	8	7	45	37	30
T10	9	3	48	39	36

Average Waiting Time = $178/10 = 17.8$ ms. Average Turnaround Time = $226/10 = 22.6$ ms.

CPU Utilization = $(48/48) \times 100\% = 100\%$ (no idle time).

Shortest Job First (SJF) Scheduling

SJF is a scheduling algorithm that selects the waiting process with the smallest burst (execution) time to execute next. This implementation is non-preemptive: once a process starts, it runs to completion.

Working Principle: Whenever the CPU is free, the scheduler scans all processes that have arrived and are waiting in the ready queue, and dispatches the one with the shortest remaining burst time. Ties are broken by arrival order.

Advantages: Provably minimizes average waiting time among non-preemptive algorithms for a given set of processes; Improves overall throughput.

Disadvantages: Requires advance knowledge (or estimation) of burst time, which is often not available in practice; Long processes may suffer starvation if short processes keep arriving.

Applications: Batch systems where burst times can be estimated or are known in advance; Cloud task schedulers optimizing for average completion time.

Calculations — SJF (non-preemptive, chosen among tasks already arrived)

Dispatch order: T1 → T5 → T2 → T8 → T10 → T6 → T4 → T9 → T3 → T7.

Task	AT	BT	CT	TAT	WT
T1	0	5	5	5	0
T5	4	2	7	3	1
T2	1	3	10	9	6
T8	7	1	11	4	3
T10	9	3	14	5	2
T6	5	4	18	13	9
T4	3	6	24	21	15
T9	8	7	31	23	16
T3	2	8	39	37	29
T7	6	9	48	42	33

Average Waiting Time = $114/10 = 11.4$ ms. Average Turnaround Time = $162/10 = 16.2$ ms.

CPU Utilization = 100%.

Round Robin (RR) Scheduling (Time Quantum = 4 ms)

Round Robin is a preemptive scheduling algorithm designed for time-sharing systems. Each process is given a fixed time slice (quantum); if it does not finish within that slice, it is preempted and placed at the back of the ready queue.

Working Principle: The ready queue is treated as a circular FIFO queue. The CPU executes the process at the head of the queue for at most one quantum. If the process needs more time, it is moved to the tail of the queue and the next process is dispatched.

Advantages: Fair allocation of CPU time to every process; Good response time, well suited to interactive/time-shared systems; No starvation.

Disadvantages: Performance is highly sensitive to the size of the time quantum; Higher average turnaround time than SJF for CPU-bound jobs; Increased context-switching overhead for small quanta.

Applications: Time-sharing operating systems; Interactive terminals and multi-user cloud task queues; Traffic-signal and round-trip resource-sharing systems.

Gantt Chart — Round Robin (Quantum = 4 ms)

T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T3	T4
0-4	4-7	7-11	11-15	15-17	17-21	21-25	25-26	26-30	30-33	33-37	37-39

Calculations — Round Robin

Task	AT	BT	CT	TAT	WT
T1	0	5	16	16	11
T2	1	3	7	6	3
T3	2	8	38	36	28
T4	3	6	40	37	31
T5	4	2	18	14	12
T6	5	4	22	17	13
T7	6	9	48	42	33
T8	7	1	27	20	19
T9	8	7	47	39	32
T10	9	3	34	25	22

Average Waiting Time = $204/10 = 20.4$ ms. Average Turnaround Time = $252/10 = 25.2$ ms.
CPU Utilization = 100%.

Comparison Table

Parameter	FCFS	SJF	Round Robin
Average Waiting Time	17.8 ms	11.4 ms (best)	20.4 ms
Average Turnaround Time	22.6 ms	16.2 ms (best)	25.2 ms
CPU Utilization	100%	100%	100%
Fairness	High	Low (short jobs favoured)	Highest
Starvation risk	None	Long tasks may starve	None
Context-switch overhead	Lowest	Low	Highest

Result

SJF produces the lowest average waiting time (11.4 ms) and turnaround time (16.2 ms) of the three algorithms; all three achieve 100% CPU utilization because tasks are purely CPU-bound with no idle gaps.

Conclusion

For a cloud environment optimizing for average completion time and cost efficiency, SJF (or its preemptive form, Shortest Remaining Time First) is recommended for batch computational tasks. Where fairness/responsiveness for many simultaneous tenants is more important than raw throughput, Round Robin remains preferable despite its higher average wait.

QUESTION 4

Problem Statement

A smart city uses sensors to detect vehicle density at an intersection. Each traffic lane is represented as a process competing for signal access. Design a scheduling strategy that minimizes vehicle waiting time. Compare the effectiveness of Round Robin and Priority Scheduling for emergency vehicles.

Aim

To design and compare a green-signal allocation strategy for four lanes, one of which carries an emergency vehicle, using Round Robin and Priority Scheduling.

Assumptions

Four lanes (North, South, East, West) are treated as processes competing for the green signal (the CPU).

An emergency vehicle is detected on the West lane 5 seconds after the cycle begins and is assigned the highest priority.

Given Data

Lane (Process)	Arrival (s)	Green Time Needed / Burst (s)	Priority
North	0	20	3
South	0	15	3
East	0	25	3
West (Emergency)	5	10	1

Round Robin (RR) Scheduling (Time Quantum = 10 s)

Round Robin is a preemptive scheduling algorithm designed for time-sharing systems. Each process is given a fixed time slice (quantum); if it does not finish within that slice, it is preempted and placed at the back of the ready queue.

Working Principle: The ready queue is treated as a circular FIFO queue. The CPU executes the process at the head of the queue for at most one quantum. If the process needs more time, it is moved to the tail of the queue and the next process is dispatched.

Advantages: Fair allocation of CPU time to every process; Good response time, well suited to interactive/time-shared systems; No starvation.

Disadvantages: Performance is highly sensitive to the size of the time quantum; Higher average turnaround time than SJF for CPU-bound jobs; Increased context-switching overhead for small quanta.

Applications: Time-sharing operating systems; Interactive terminals and multi-user cloud task queues; Traffic-signal and round-trip resource-sharing systems.

Round Robin (Quantum = 10 s) — Signal Allocation

N	S	E	W	N	S	E	E
0-10	10-20	20-30	30-40	40-50	50-55	55-65	65-70

The emergency vehicle (West) arrives at $t = 5$ s but, because RR treats every lane equally and does not consider urgency, it is not granted a green signal until $t = 30$ s — a delay of 25 seconds. This is unacceptable for an emergency vehicle.

Priority Scheduling

Priority Scheduling assigns each process a priority number; the CPU is allocated to the process with the highest priority (in this document, a lower numeric value denotes a higher priority). This implementation is non-preemptive unless stated otherwise.

Working Principle: The scheduler always picks the highest-priority process among those that have arrived and are waiting. Processes with equal priority are served in FCFS order. A preemptive variant interrupts a running lower-priority process the instant a higher-priority process arrives.

Advantages: Critical or time-sensitive tasks receive prompt service; Flexible — priority can reflect urgency, resource needs, or business rules.

Disadvantages: Low-priority processes may suffer indefinite starvation; Requires a fair method (e.g., aging) to bound waiting time.

Applications: Emergency/critical-task handling (hospitals, satellites); Real-time and embedded systems; Multilevel service-tier systems (e.g., premium vs standard users).

Priority Scheduling (Non-preemptive)

Non-preemptive priority still cannot interrupt the lane already showing green: North (already running when West arrives) continues to completion, so West must wait until North finishes at $t = 20$ s.

N	W	S	E
0-20	20-30	30-45	45-70

West's response time improves to $20 - 5 = 15$ seconds — better than RR, but still not immediate.

Recommended Strategy — Preemptive Priority Scheduling

If the signal controller is allowed to preempt the current green phase the instant an emergency vehicle is detected, North's phase is cut short at $t = 5$ s and West is granted an immediate green (5-15 s), giving a response time of 0 seconds.

N	W	N(resume)	S	E
0-5	5-15	15-30	30-45	45-70

Comparison Table

Parameter	Round Robin (q=10s)	Non-preemptive Priority	Preemptive Priority
Emergency response time	25 s	15 s	0 s
Fairness to regular lanes	High	Medium	Medium
Complexity	Low	Medium	High (needs preemption logic)
Best use case	Normal, balanced traffic	Moderate urgency	Emergency vehicle priority

Result

Round Robin minimizes average waiting time fairly across ordinary lanes but cannot react to emergencies. Non-preemptive Priority Scheduling improves emergency response but is still bounded by the current green phase. Preemptive Priority Scheduling gives an emergency vehicle an immediate (zero-second) response.

Conclusion

The recommended strategy is a hybrid: Round Robin (or density-weighted RR) for normal traffic flow, combined with a preemptive priority override triggered by emergency-vehicle sensors, so

that ordinary lanes are served fairly while emergency vehicles always receive an immediate green signal.

QUESTION 5

Problem Statement

During an online examination, thousands of students submit answers simultaneously. The server processes requests based on arrival times and submission priorities. Construct a process scheduling table for eight requests and calculate completion time, turnaround time, and waiting time using FCFS and SJF scheduling.

Aim

To model exam-submission processing as eight scheduling requests and compare FCFS and SJF for completion, turnaround and waiting times.

Assumptions

Eight submission requests (R1-R8) arrive over an 8-second window with the processing (burst) times shown below.

Given Data

Request	Arrival (s)	Burst (s)
R1	0	3
R2	1	5
R3	2	2
R4	3	7
R5	4	1
R6	5	4
R7	6	6
R8	7	2

First-Come, First-Served (FCFS) Scheduling

FCFS is a non-preemptive CPU scheduling algorithm in which the process that requests the CPU first is allocated the CPU first. Processes are executed strictly in the order of their arrival, using a simple FIFO (First-In-First-Out) ready queue.

Working Principle: When the CPU becomes free, the process at the head of the ready queue is dispatched and runs to completion (or until it needs I/O) before the next process in the queue is selected. No process can pre-empt a running process.

Advantages: Simple to understand and implement; Fair in the sense that every process is served in arrival order; No starvation, since every process is eventually served.

Disadvantages: Leads to the convoy effect, where short processes wait behind long ones; Average waiting time is generally high compared to SJF; Not suitable for time-sharing / interactive systems.

Applications: Batch processing systems; Print queues and simple task queues; Systems where arrival order fairness matters more than turnaround time.

Gantt Chart — FCFS

R1	R2	R3	R4	R5	R6	R7	R8
0-3	3-8	8-10	10-17	17-18	18-22	22-28	28-30

Calculations — FCFS

Request	AT	BT	CT	TAT	WT
R1	0	3	3	3	0
R2	1	5	8	7	2
R3	2	2	10	8	6
R4	3	7	17	14	7
R5	4	1	18	14	13
R6	5	4	22	17	13
R7	6	6	28	22	16
R8	7	2	30	23	21

Average Waiting Time = $78/8 = 9.75$ s. Average Turnaround Time = $108/8 = 13.5$ s.

Shortest Job First (SJF) Scheduling

SJF is a scheduling algorithm that selects the waiting process with the smallest burst (execution) time to execute next. This implementation is non-preemptive: once a process starts, it runs to completion.

Working Principle: Whenever the CPU is free, the scheduler scans all processes that have arrived and are waiting in the ready queue, and dispatches the one with the shortest remaining burst time. Ties are broken by arrival order.

Advantages: Provably minimizes average waiting time among non-preemptive algorithms for a given set of processes; Improves overall throughput.

Disadvantages: Requires advance knowledge (or estimation) of burst time, which is often not available in practice; Long processes may suffer starvation if short processes keep arriving.

Applications: Batch systems where burst times can be estimated or are known in advance; Cloud task schedulers optimizing for average completion time.

Gantt Chart — SJF (non-preemptive)

R1	R3	R5	R6	R8	R2	R7	R4
0-3	3-5	5-6	6-10	10-12	12-17	17-23	23-30

Calculations — SJF

Request	AT	BT	CT	TAT	WT
R1	0	3	3	3	0
R3	2	2	5	3	1
R5	4	1	6	2	1
R6	5	4	10	5	1
R8	7	2	12	5	3
R2	1	5	17	16	11
R7	6	6	23	17	11
R4	3	7	30	27	20

Average Waiting Time = $48/8 = 6$ s. Average Turnaround Time = $78/8 = 9.75$ s.

Comparison Table

Parameter	FCFS	SJF
Average Waiting Time	9.75 s	6 s
Average Turnaround Time	13.5 s	9.75 s
Fairness	High	Favours short submissions
Starvation risk	None	Large submissions (e.g. R4) may be delayed

Result

SJF reduces average waiting time from 9.75 s to 6 s and average turnaround time from 13.5 s to 9.75 s compared with FCFS.

Conclusion

For a high-volume online exam server, SJF (or a size-aware variant) processes the majority of (typically small) submissions faster and reduces overall queueing delay, but the server should reserve a fraction of capacity or use aging so that large submissions such as R4 are not starved during peak load.

QUESTION 6

Problem Statement

A factory assembly line consists of multiple robotic arms, each performing a task. If one robot fails, the operating system must reschedule pending tasks. Analyse how process states (Ready, Running, Waiting, Terminated) change during task execution and explain how scheduling ensures continuous production.

Aim

To model each robotic-arm task as a process and trace its state transitions during normal operation and during a robot failure/reschedule event.

Assumptions

Four robotic arms (R1-Weld, R2-Paint, R3-Assemble, R4-Inspect) each execute one task per production cycle.

R2 (Paint) suffers a fault at $t = 12$ and must be rescheduled to a backup arm.

Theory — Process State Model

Every task passes through the standard five-state process model used by the operating system scheduler: New (task created/queued for a cycle), Ready (waiting for the arm/CPU to be free), Running (actively executing on an arm), Waiting/Blocked (suspended for a resource, part feed, or event such as a fault), and Terminated (task completed).

Process State Transition Diagram

	NEW	
	↓ (admitted)	
	READY	
	↑ (dispatch / time-out)	
	RUNNING	
(I/O or event wait) ↙		↘ (exit)
WAITING		TERMINATED
(event completion → Ready)		

State Trace During a Robot Failure

Time	R1-Weld	R2-Paint	R3-Assemble	R4-Inspect
t=0	Ready	Ready	Ready	Ready
t=1	Running	Ready	Ready	Ready
t=5	Terminated	Running	Ready	Ready
t=12 (fault)	Ready (next unit)	Waiting (fault → OS reschedules)	Running	Ready
t=13	Running	Waiting (task moved to backup arm queue)	Terminated	Ready
t=18	Terminated	Running (on backup arm)	Ready (next unit)	Running
t=24	Ready (next unit)	Terminated	Running	Terminated

Scheduling Flow Ensuring Continuous Production

Task created for production cycle (New)
↓
Task enters Ready queue for its assigned arm
↓
Scheduler dispatches task → Running
↓
Fault detected on arm? Yes → move task to Waiting, log fault, reassign to backup arm's Ready queue
↓
No fault → task completes → Terminated
↓
Scheduler dispatches next queued task → line continues

Result

Robotic-arm tasks move through the classic New → Ready → Running → (Waiting) → Terminated cycle. When R2 (Paint) faults at t = 12, its in-progress task moves to Waiting rather than being lost, and the OS scheduler reassigns it to a Ready queue for a backup arm instead of stalling the whole line.

Conclusion

By keeping every task's state explicit and re-queuing rather than discarding work on failure, the scheduler allows the assembly line to continue production with only the faulted arm's throughput affected, demonstrating why explicit process-state management (not just raw task lists) is essential for fault-tolerant industrial automation.

QUESTION 7

Problem Statement

A video streaming service processes user requests for video buffering. Premium users are assigned higher priority than standard users. Given a set of user requests with arrival and burst times, implement Priority Scheduling and evaluate fairness, starvation, and throughput.

Aim

To evaluate the impact of Priority Scheduling on premium vs. standard streaming users in terms of waiting time, fairness, and starvation risk.

Assumptions

Six buffering requests; Premium = priority 1, Standard = priority 2.

Given Data

User	Type	Arrival (s)	Buffer/Burst (s)	Priority
U1	Premium	0	4	1
U2	Standard	1	3	2
U3	Premium	2	2	1
U4	Standard	3	5	2
U5	Standard	4	2	2
U6	Premium	5	3	1

Priority Scheduling

Priority Scheduling assigns each process a priority number; the CPU is allocated to the process with the highest priority (in this document, a lower numeric value denotes a higher priority). This implementation is non-preemptive unless stated otherwise.

Working Principle: The scheduler always picks the highest-priority process among those that have arrived and are waiting. Processes with equal priority are served in FCFS order. A preemptive variant interrupts a running lower-priority process the instant a higher-priority process arrives.

Advantages: Critical or time-sensitive tasks receive prompt service; Flexible — priority can reflect urgency, resource needs, or business rules.

Disadvantages: Low-priority processes may suffer indefinite starvation; Requires a fair method (e.g., aging) to bound waiting time.

Applications: Emergency/critical-task handling (hospitals, satellites); Real-time and embedded systems; Multilevel service-tier systems (e.g., premium vs standard users).

Gantt Chart — Priority Scheduling

U1	U3	U6	U2	U4	U5
0-4	4-6	6-9	9-12	12-17	17-19

Calculations

User	Type	AT	BT	CT	TAT	WT
U1	Premium	0	4	4	4	0
U3	Premium	2	2	6	4	2
U6	Premium	5	3	9	4	1
U2	Standard	1	3	12	11	8
U4	Standard	3	5	17	14	9
U5	Standard	4	2	19	15	13

Average Waiting Time — Premium = $(0+2+1)/3 = 1$ s.

Average Waiting Time — Standard = $(8+9+13)/3 = 10$ s.

Fairness, Starvation, and Throughput Evaluation

- Fairness: heavily skewed toward premium users — a 10x difference in average waiting time between tiers.
- Starvation: standard users are not fully starved in this run, but if premium requests continue to arrive faster than the server can drain them, standard requests could be delayed indefinitely — a real starvation risk under sustained premium load.
- Throughput: total requests processed and total server-busy time are unchanged (19 s to serve all six requests) — Priority Scheduling redistributes waiting time rather than harming overall throughput.

Result

Priority Scheduling reduces premium-user average wait to 1 s versus 10 s for standard users, i.e., it delivers a clearly superior quality of experience to the paying/priority tier without reducing total system throughput.

Conclusion

Priority Scheduling is appropriate for a tiered streaming service, but an aging mechanism (gradually raising a standard request's effective priority the longer it waits) should be added to guarantee an upper bound on standard-user wait time and avoid starvation under heavy premium load.

QUESTION 8

Problem Statement

A bank processes ATM withdrawals, deposits, and fund transfers concurrently. Each transaction is treated as a process with varying execution times. Using Round Robin Scheduling (time quantum = 4 ms), determine the execution order and calculate average turnaround and waiting times for ten transactions.

Aim

To schedule ten concurrent ATM transactions using Round Robin (quantum = 4 ms) and compute average turnaround and waiting time.

Assumptions

Ten transactions (withdrawals, deposits, transfers) arrive over a 9 ms window with varying execution times as given below.

Given Data

Transaction	Arrival (ms)	Burst (ms)
Tx1	0	6
Tx2	1	3
Tx3	2	9
Tx4	3	5
Tx5	4	2
Tx6	5	7
Tx7	6	4
Tx8	7	8
Tx9	8	3
Tx10	9	5

Round Robin (RR) Scheduling (Time Quantum = 4 ms)

Round Robin is a preemptive scheduling algorithm designed for time-sharing systems. Each process is given a fixed time slice (quantum); if it does not finish within that slice, it is preempted and placed at the back of the ready queue.

Working Principle: The ready queue is treated as a circular FIFO queue. The CPU executes the process at the head of the queue for at most one quantum. If the process needs more time, it is moved to the tail of the queue and the next process is dispatched.

Advantages: Fair allocation of CPU time to every process; Good response time, well suited to interactive/time-shared systems; No starvation.

Disadvantages: Performance is highly sensitive to the size of the time quantum; Higher average turnaround time than SJF for CPU-bound jobs; Increased context-switching overhead for small quanta.

Applications: Time-sharing operating systems; Interactive terminals and multi-user cloud task queues; Traffic-signal and round-trip resource-sharing systems.

Gantt Chart — Round Robin (Quantum = 4 ms)

Tx1	Tx2	Tx3	Tx4	Tx1	Tx5	Tx6	Tx7	Tx8	Tx9	Tx10	Tx3
0-4	4-7	7-11	11-15	15-17	17-19	19-23	23-27	27-31	31-34	34-38	38-42

Calculations

Transaction	AT	BT	CT	TAT	WT
Tx1	0	6	17	17	11
Tx2	1	3	7	6	3
Tx3	2	9	52	50	41
Tx4	3	5	43	40	35
Tx5	4	2	19	15	13
Tx6	5	7	46	41	34
Tx7	6	4	27	21	17
Tx8	7	8	50	43	35
Tx9	8	3	34	26	23
Tx10	9	5	51	42	37

Average Waiting Time = $249/10 = 24.9$ ms.

Average Turnaround Time = $301/10 = 30.1$ ms.

Result

All ten ATM transactions complete by $t = 52$ ms with an average turnaround time of 30.1 ms and average waiting time of 24.9 ms; the 4 ms quantum causes several transactions (Tx1, Tx3, Tx4, Tx6, Tx8, Tx10) to be preempted and re-queued multiple times.

Conclusion

Round Robin guarantees that no single transaction (e.g., a large transfer) can block smaller withdrawals/deposits indefinitely, which is important for ATM fairness, but the resulting turnaround times are higher than an SJF-style schedule would give. In practice banks combine RR-style time-slicing with priority boosts for time-critical transactions (e.g., fraud holds) on top of this fairness baseline.

QUESTION 9

Problem Statement

An autonomous vehicle executes multiple processes simultaneously: obstacle detection, GPS navigation, speed control, and lane detection. Identify the process states of each task at different time intervals and justify the use of preemptive scheduling in ensuring passenger safety.

Aim

To trace the process states of four concurrently-executing autonomous-vehicle tasks and justify preemptive scheduling for safety-critical control.

Assumptions

Obstacle Detection = highest priority (safety-critical, must preempt everything). Speed Control = priority 2. Lane Detection = priority 3. GPS Navigation = lowest priority (non-time-critical route updates).

A pedestrian is detected by the obstacle sensor at $t = 6$, requiring an immediate preemption.

Given Data

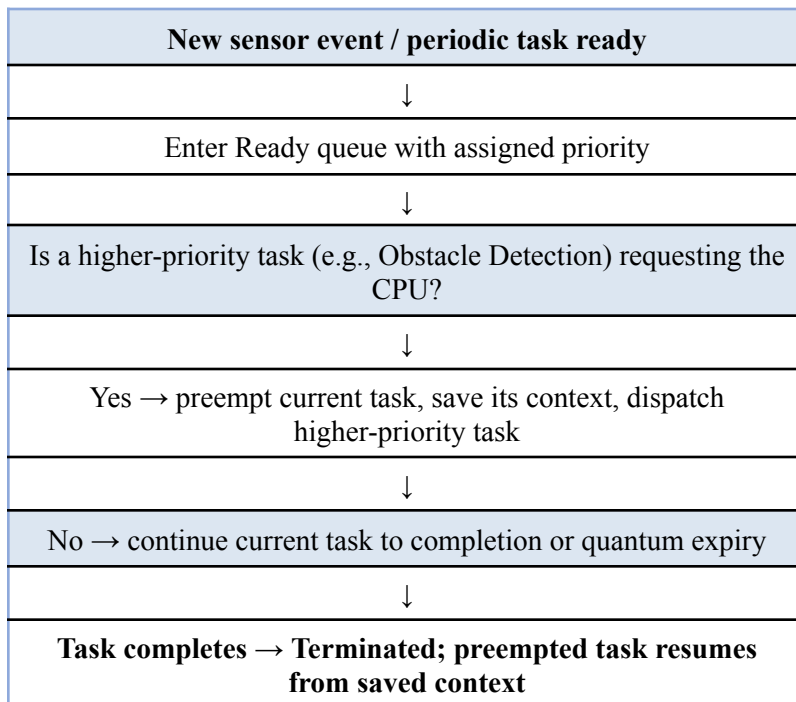
Task (Process)	Priority	Nominal Burst per cycle (ms)
Obstacle Detection	1 (highest)	3
Speed Control	2	4
Lane Detection	3	5
GPS Navigation	4 (lowest)	6

Process State Trace

Time	Obstacle Detection	Speed Control	Lane Detection	GPS Navigation
$t=0$	Running	Ready	Ready	Ready
$t=3$	Terminated (cycle done)	Running	Ready	Ready
$t=6$ (pedestrian detected)	Running (preempts Speed Control)	Ready (preempted)	Ready	Waiting
$t=9$	Terminated	Running (resumed)	Ready	Waiting
$t=11$	Ready	Terminated	Running	Waiting

Time	Obstacle Detection	Speed Control	Lane Detection	GPS Navigation
t=16	Ready	Ready	Terminated	Running
t=22	Ready (next cycle)	Ready (next cycle)	Ready (next cycle)	Terminated

Scheduling Decision Flow



Result

When the obstacle sensor detects a pedestrian at $t = 6$ while Speed Control is running, the scheduler preempts Speed Control (returning it to Ready with its context saved) and immediately dispatches Obstacle Detection, which completes at $t = 9$ before Speed Control resumes.

Conclusion

Non-preemptive scheduling would force Obstacle Detection to wait until the currently running task finished, which is unacceptable when milliseconds determine whether a collision is avoided. Preemptive, priority-based scheduling is therefore essential in autonomous vehicles: safety-critical perception and braking tasks must always be able to interrupt lower-priority tasks such as GPS route recalculation.

QUESTION 10

Problem Statement

A satellite control center monitors communication, navigation, power management, and telemetry systems. Communication tasks have the highest priority during emergencies. Simulate process execution using Priority Scheduling and analyze the impact of process starvation. Propose a solution using aging techniques to improve fairness.

Aim

To simulate satellite subsystem tasks under Priority Scheduling, demonstrate starvation of a low-priority task, and design an aging-based fix.

Assumptions

Four recurring subsystem tasks; Communication runs in emergency mode and repeatedly re-arrives with priority 1, crowding out the lowest-priority Telemetry task.

Given Data

Task	Base Priority	Burst per instance (ms)	Arrival Pattern
Communication (emergency)	1 (highest)	5	Repeats every 6 ms
Power Management	2	6	Once, $t=0$
Navigation	3	8	Once, $t=0$
Telemetry	4 (lowest)	10	Once, $t=0$

Priority Scheduling

Priority Scheduling assigns each process a priority number; the CPU is allocated to the process with the highest priority (in this document, a lower numeric value denotes a higher priority). This implementation is non-preemptive unless stated otherwise.

Working Principle: The scheduler always picks the highest-priority process among those that have arrived and are waiting. Processes with equal priority are served in FCFS order. A preemptive variant interrupts a running lower-priority process the instant a higher-priority process arrives.

Advantages: Critical or time-sensitive tasks receive prompt service; Flexible — priority can reflect urgency, resource needs, or business rules.

Disadvantages: Low-priority processes may suffer indefinite starvation; Requires a fair method (e.g., aging) to bound waiting time.

Applications: Emergency/critical-task handling (hospitals, satellites); Real-time and embedded systems; Multilevel service-tier systems (e.g., premium vs standard users).

Simulation — Starvation Without Aging

Because a fresh, higher-priority Communication instance arrives every 6 ms, the non-preemptive scheduler keeps selecting Communication (and then Power Management/Navigation) ahead of Telemetry every time the CPU frees up, so Telemetry never gets dispatched during the observation window.

Time Window	Task Dispatched	Telemetry Status
0-5	Communication #1	Ready (waiting)
5-11	Power Management	Ready (waiting)
11-17	Communication #2 (re-armed)	Ready (waiting)
17-25	Navigation	Ready (waiting)
25-30	Communication #3	Ready (waiting, 30 ms elapsed — starving)

Proposed Solution — Aging

Aging gradually increases the effective priority (decreases the priority number) of a task the longer it waits, so that no task can be delayed indefinitely.

Aging Rule Applied

Effective Priority = Base Priority – floor(Waiting Time / 5 ms), with a floor of 1 (highest).

Waiting Time (ms)	Telemetry Base Priority	Aged Effective Priority
0-4	4	4
5-9	4	3
10-14	4	2
15+	4	1 (now equals Communication — guaranteed dispatch)

Result

Under plain Priority Scheduling, Telemetry is repeatedly overtaken by recurring high-priority Communication instances and other higher-priority tasks, and its waiting time grows without bound — a textbook case of starvation.

Conclusion

By applying aging (raising Telemetry's effective priority for every 5 ms it waits), Telemetry reaches priority 1 after 15 ms of waiting and is guaranteed dispatch at the next scheduling decision, bounding its maximum wait while still letting genuine emergencies (Communication) be served promptly. Aging is therefore recommended for all satellite subsystem scheduling to balance responsiveness with fairness.

- 11) A set of processes arrive at time 0 with the following burst times :

Process	P ₁	P ₂	P ₃	P ₄
Burst Time (ms)	10	4	6	2

- Calculate Waiting Time and Turnaround Time for each process using SJF (Non-Preemptive).
- Compute the average waiting time and average turnaround time.
- Interpret whether SJF improves CPU utilization compared to FCFS for this workload.

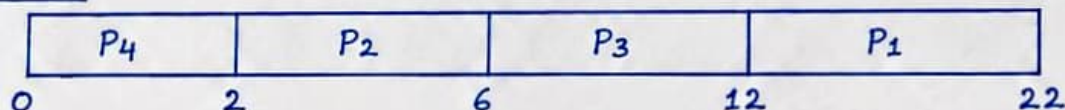
Solution :

a) SJF (Non-Preemptive) Scheduling

→ Since all processes arrive at time 0, SJF (Non-Preemptive) will execute processes in increasing order of Burst Time.

Order of execution : P₄ (2) → P₂ (4) → P₃ (6) → P₁ (10)

• Gantt Chart :



• Process Table :

Process	Burst Time (ms)	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P ₁	10	22	22 - 0 = 22	22 - 10 = 12
P ₂	4	6	6 - 0 = 6	6 - 4 = 2
P ₃	6	12	12 - 0 = 12	12 - 6 = 6
P ₄	2	2	2 - 0 = 2	2 - 2 = 0

b) Average Waiting Time (AWT) = $\frac{12 + 2 + 6 + 0}{4} = \frac{20}{4} = \boxed{5 \text{ ms}}$

Average Turnaround Time (ATT) = $\frac{22 + 6 + 12 + 2}{4} = \frac{42}{4} = \boxed{10.5 \text{ ms}}$

c) Interpretation :

SJF schedules the shortest jobs first, which reduces the average waiting time and turnaround time compared to FCFS. Since processes spend less time waiting in the ready queue, CPU remains busy for longer periods, thereby improving CPU utilization for this workload.

13) A system has the following process behavior :

- Each process spends 20% of its time using the CPU and 80% waiting for I/O.
- Degree of multiprogramming = 4 processes

The CPU utilization is given by :

$$U = 1 - p^n \quad \text{where,}$$

p = Probability that a process is waiting for I/O
 n = Number of processes in memory

- Calculate the CPU Utilization.
- Compute the CPU Utilization if the degree of multiprogramming increases to 6.
- Interpret which configuration provides better system performance and explain why.

Solution :

Given,

CPU time = 20%.

I/O waiting time = 80%.

So, p = Probability of waiting for I/O = 80% = 0.8

- a) For degree of multiprogramming (n) = 4

Using $U = 1 - p^n$

$$U = 1 - (0.8)^4$$

$$U = 1 - 0.4096$$

$$U = 0.5904$$

$$U = 59.04\% \quad (\text{CPU Utilization})$$

- b) For degree of multiprogramming (n) = 6

$$U = 1 - p^n$$

$$U = 1 - (0.8)^6$$

$$U = 1 - 0.262144$$

$$U = 0.737856$$

$$U = 73.79\% \quad (\text{CPU Utilization})$$

- c) Interpretation :

When the degree of multiprogramming increases from 4 to 6, CPU utilization increases from 59.04% to 73.79%.

This configuration with 6 processes provides better system performance because more processes keep the CPU busy most of the time while others wait for I/O, reducing CPU idle time and increasing overall utilization.

12) Consider the following processes :

Process	P ₁	P ₂	P ₃	P ₄
Burst Time (ms)	20	5	13	8

Time Quantum = 4 ms

- Construct the Gantt Chart for Round Robin Scheduling.
- Calculate the Waiting Time and Turnaround Time of each process.
- Analyze how changing the time quantum to 10 ms would affect CPU utilization and context switching.

Solution :

a) Gantt Chart for Round Robin Scheduling (Time Quantum = 4 ms)



Step by Step Explanation :

- | | |
|--|---|
| 1. 0-4 : P ₁ executes for 4 ms (rem = 16) | 8. 28-32 : P ₄ executes for 4 ms (finishes) |
| 2. 4-8 : P ₂ executes for 4 ms (rem = 1) | 9. 32-36 : P ₁ executes for 4 ms (rem = 8) |
| 3. 8-12 : P ₃ executes for 4 ms (rem = 9) | 10. 36-40 : P ₃ executes for 4 ms (rem = 1) |
| 4. 12-16 : P ₄ executes for 4 ms (rem = 4) | 11. 40-44 : P ₁ executes for 4 ms (rem = 4) |
| 5. 16-20 : P ₁ executes for 4 ms (rem = 12) | 12. 44-45 : P ₃ executes for 1 ms (finishes) |
| 6. 20-24 : P ₂ executes for 1 ms (finishes) | 13. 45-49 : P ₁ executes for 4 ms (finishes) |
| 7. 24-28 : P ₃ executes for 4 ms (rem = 5) | |

b) Waiting Time and Turnaround Time

- Completion Time (CT) from Gantt Chart :

$$P_1 \rightarrow CT = 49$$

$$P_2 \rightarrow CT = 20$$

$$P_3 \rightarrow CT = 45$$

$$P_4 \rightarrow CT = 32$$

- Calculation Table :

Process	Burst Time (BT)	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P ₁	20	49	$49 - 0 = 49$	$49 - 20 = 29$
P ₂	5	20	$20 - 0 = 20$	$20 - 5 = 15$
P ₃	13	45	$45 - 0 = 45$	$45 - 13 = 32$
P ₄	8	32	$32 - 0 = 32$	$32 - 8 = 24$

$$\text{Average Waiting Time (AWT)} = \frac{(29 + 15 + 32 + 24)}{4} = \frac{100}{4} = \boxed{25 \text{ ms}}$$

$$\text{Average Turnaround Time (ATT)} = \frac{(49 + 20 + 45 + 32)}{4} = \frac{146}{4} = \boxed{36.5 \text{ ms}}$$

c) Effect of Changing Time Quantum to 10 ms

Analysis :

- With Time Quantum = 10 ms, each process will get more CPU time in one go.
- Number of context switches will decrease because preemptions are reduced.
- This reduces context switching overhead and improves CPU utilization.
- Waiting time may increase for short processes that arrive after long processes.
- Throughput may improve slightly due to fewer context switches.
- Overall, system becomes more efficient, but response time for some processes may increase.